



IdentityWallet

PROJECT REPORT

Digital Identity Wallet

A privacy-preserving digital identity management system with verifiable credentials, selective disclosure and an encrypted document vault

Java 17

Spring Boot 3.3

React 18

ECDSA P-256

AES-256-GCM

TOTP 2FA

Submitted by	Cecil Baraik
Roll number	2025-BSCIT/004A
Programme	B.Sc. Information Technology
Institution	Amity University Jharkhand
Project guide	_____
Session	2026

Certificate

This is to certify that the project titled "**Digital Identity Wallet — Digital Identity Management**" is a bonafide work carried out by **Cecil Baraik** (Roll No. **2025-BSCIT/004A**), a student of **B.Sc. Information Technology, Amity University Jharkhand**, under my supervision and guidance.

The project has been completed satisfactorily and the work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma.

Project Guide

Name: _____

Date: _____

Head of Department

Name: _____

Date: _____

Declaration

I, **Cecil Baraik**, hereby declare that the project report entitled "**Digital Identity Wallet — Digital Identity Management**" submitted to Amity University Jharkhand is a record of original work done by me. The design, source code, testing and documentation presented here were produced for this project, and all external concepts, standards and libraries are acknowledged in the References.

Signature of Student
Cecil Baraik
Roll No. 2025-BSCIT/004A

Place: Ranchi
Date: _____

Acknowledgement

I express my sincere gratitude to my project guide for constant support and valuable suggestions throughout this work, and to the faculty of the Department of Information Technology, Amity University Jharkhand, for the knowledge and facilities provided. I also thank my family and friends for their encouragement. Finally, I acknowledge the open-source communities behind Java, Spring Boot, React and the IETF / W3C identity standards whose work made this project possible.

Abstract

Every day people prove who they are — at a bank, a cinema, a college office or a new job — by handing over photocopies of identity documents. Each copy reveals far more than needed (a cinema only needs to know you are over 18, but sees your full birth date, address and ID number), copies are easy to forge, and the person has no idea where their data goes afterwards.

IdentityWallet is a full-stack web application that solves this with **verifiable digital credentials**.

Trusted organisations (issuers) sign credentials with **ECDSA P-256**; the holder stores them in an **encrypted wallet** and shares them through a QR code or link. Using **selective disclosure** — each claim is salted and hashed (SHA-256) before signing — the holder can reveal only chosen details, and even derived facts such as *"age over 18"* without revealing the date of birth. A verifier checks five things automatically: trusted issuer, issuer signature, claim integrity, holder consent and status (expiry/revocation).

The system also includes an **AES-256-GCM encrypted document vault** with envelope encryption, **two-factor sign-in (TOTP, RFC 6238)**, PBKDF2 password hashing, JWT sessions, account lockout, rate limiting, consent logs showing every access, and a **tamper-evident hash-chained audit log**. It is built with **Java 17, Spring Boot 3.3, Spring Data JPA, H2/MySQL and React 18**, and is validated by **8 automated tests** including forgery and tampering attempts, all of which pass.

Keywords: digital identity, verifiable credentials, selective disclosure, ECDSA, AES-GCM, envelope encryption, TOTP, privacy by design, Spring Boot, React.

Table of Contents

1 Introduction

- 1.1 Background
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope

2 Literature Survey and Existing Systems

- 2.1 Existing approaches
- 2.2 Standards and concepts studied
- 2.3 What this project adds

3 System Requirements and Analysis

- 3.1 Functional requirements
- 3.2 Non-functional requirements
- 3.3 Hardware and software requirements
- 3.4 Feasibility

4 System Design

- 4.1 Architecture
- 4.2 Modules
- 4.3 Credential lifecycle
- 4.4 Selective disclosure design
- 4.5 Key management (envelope encryption)
- 4.6 Database design
- 4.7 REST API design

5 Implementation

- 5.1 Technology stack
- 5.2 Project structure
- 5.3 Issuing a signed credential
- 5.4 Creating a share (holder consent)
- 5.5 Verification
- 5.6 Encryption and key management
- 5.7 Authentication and session security
- 5.8 Tamper-evident audit log
- 5.9 Frontend highlights

6 Security Analysis

- 6.1 Threat model (STRIDE)
- 6.2 OWASP Top 10 mapping

7 Testing

- 7.1 Test strategy
- 7.2 Automated test cases
- 7.3 Test execution result
- 7.4 Manual test summary

8 Results and Screenshots

9 Conclusion and Future Scope

- 9.1 Conclusion
- 9.2 Limitations
- 9.3 Future scope

References

1 Introduction

1.1 Background

Digital identity is the set of information that represents a person online — name, date of birth, government ID, education, employment and so on. Today this information is mostly shared as scanned images or photocopies. Once shared, the owner loses control of it: the receiver keeps a full copy forever, may store it insecurely, and cannot easily check whether the document is genuine.

Modern identity research (the W3C *Verifiable Credentials* model and the IETF *SD-JWT* draft) proposes a better approach: an organisation **issues** a digitally signed credential to a person, the person **holds** it in a wallet, and any **verifier** can check the signature mathematically. This project implements that model as a working application.

1.2 Problem Statement

1. **Over-sharing:** a photocopy reveals every detail on the card, even when only one fact is needed.
2. **Forgery:** paper or image copies can be edited easily, and receivers cannot verify them quickly.
3. **No consent or tracking:** once a copy is handed over, the owner cannot limit how long it is used or see who looked at it.
4. **Insecure storage:** identity scans kept in email, phone galleries or unencrypted databases are a frequent target of data breaches.
5. **No revocation:** a cancelled licence or expired card keeps "working" as long as the copy exists.

1.3 Objectives

- Allow trusted issuers to create **digitally signed credentials** that cannot be forged or altered.
- Let holders **share only selected details** (selective disclosure), including privacy-safe derived claims like *age over 18*.
- Give holders **consent controls**: recipient, purpose, expiry time, view limit, stop sharing, and an access log.
- Provide **instant verification without an account** via link, QR code or a saved signed file.
- Store personal documents in an **encrypted vault** where the database alone reveals nothing.
- Protect accounts with **strong password hashing, two-factor authentication, lockout and rate limiting**.
- Keep a **tamper-evident audit trail** of every security-relevant action.

1.4 Scope

The system covers three roles — **Wallet holder**, **Issuer staff** and **Administrator** — plus anonymous **Verifiers**. It supports six built-in credential templates (National ID, Driving Licence, Student ID, Employment Certificate, Health Insurance, Address Proof) and custom claims. It runs as a web application on a local machine or server, with H2 for development and MySQL for deployment.

Out of scope for this version: mobile native apps, hardware-backed keys, and interoperability with other wallet vendors (discussed in Future Scope).

2 Literature Survey and Existing Systems

2.1 Existing approaches

System / approach	How it works	Limitations
Photocopies / scanned PDFs	Person sends an image of the card	Reveals everything, easy to forge, no revocation, no consent
Centralised document lockers (e.g. cloud drives)	Documents stored in a provider account and shared as files	Still all-or-nothing sharing; trust depends on the provider; files can be copied freely
Government e-KYC APIs	Verifier calls a central server with the citizen's ID	Requires sharing the ID number; central server sees every verification; needs integration
Blockchain identity systems	Credential hashes anchored on a public ledger	Heavy infrastructure; personal data must stay off-chain anyway; slower
W3C Verifiable Credentials + SD-JWT (this project)	Issuer signs a credential; holder presents selected claims; verifier checks signatures	Requires a trust registry of issuers (provided here)

2.2 Standards and concepts studied

- **W3C Verifiable Credentials Data Model** — defines the issuer → holder → verifier triangle used in this project.
- **IETF SD-JWT (Selective Disclosure for JWTs)** — the salted-hash disclosure technique implemented in `SelectiveDisclosure.java`.
- **Decentralized Identifiers (DIDs)** — identifiers derived from public keys; implemented as `did:dw:<hash of public key>`.
- **NIST FIPS 186 (ECDSA)** and **NIST SP 800-38D (AES-GCM)** — signature and authenticated-encryption algorithms.
- **RFC 6238 (TOTP)** — time-based one-time passwords for two-factor sign-in.
- **RFC 8018 (PBKDF2)** and **OWASP Password Storage Cheat Sheet** — password hashing parameters.
- **RFC 7519 (JWT)** — session tokens.
- **India's DPDP Act 2023 / GDPR principles** — data minimisation, purpose limitation and consent, which guided the sharing design.

2.3 What this project adds

Compared with document lockers, IdentityWallet shares **facts instead of files**, lets the holder prove derived facts without revealing the source data, binds every share to a recipient with the holder's own signature, and logs every view. Compared with blockchain systems, it achieves integrity with standard digital signatures and a trust registry, keeping the design simple and fast.

3 System Requirements and Analysis

3.1 Functional requirements

ID	Requirement	Role
FR1	Register a wallet (name, email, password); a key pair and DID are generated automatically	Holder
FR2	Sign in with password, and with an authenticator code if 2FA is on	All
FR3	View credentials in the wallet with status (Active / Expired / Revoked)	Holder
FR4	View credential details, issuer signature status and the signed payload	Holder
FR5	Create a share: choose claims, recipient, purpose, expiry and view limit; get a link + QR	Holder
FR6	See all shares, their views and access log; stop sharing at any time	Holder
FR7	Store, edit, download and delete documents in an encrypted vault	Holder
FR8	Enable / disable two-factor authentication; change password	All
FR9	View personal activity log	All
FR10	Issue a credential to a holder from a template or with custom claims	Issuer
FR11	List issued credentials and revoke one with a reason	Issuer
FR12	Register issuer organisations (with signing keys and first staff login); suspend / restore	Admin
FR13	Enable / disable user accounts; view platform statistics and the audit log; verify the hash chain	Admin
FR14	Verify a share link, scanned QR code or uploaded presentation file without an account	Verifier

3.2 Non-functional requirements

Category	Requirement
Security	No plaintext passwords, keys or vault data at rest; signed credentials; 2FA; lockout; rate limits; security headers
Privacy	Data minimisation via selective disclosure; consent per share; access logs
Integrity	Any modification of a credential, claim or audit event must be detectable
Usability	Clean responsive UI (desktop and mobile), light and dark mode, demo accounts
Performance	Verification in well under one second; password hashing tuned to ~0.4 s to resist brute force
Portability	Runs on any OS with Java 17+ and Node 18+; H2 by default, MySQL via a profile
Maintainability	Layered architecture (controller → service → repository), automated tests

3.3 Hardware and software requirements

Item	Minimum
Processor / RAM	Dual-core CPU, 4 GB RAM (8 GB recommended)
Operating system	Windows 10/11, Linux or macOS
Java	JDK 17 or newer (tested on JDK 21)
Build tools	Maven 3.9 (downloaded automatically by <code>mvnw.cmd</code>), Node.js 18+ and npm
Database	H2 (embedded, default) or MySQL 8
Browser	Any modern browser (Chrome, Edge, Firefox); camera optional for QR scanning
IDE (optional)	IntelliJ IDEA / VS Code

3.4 Feasibility

- **Technical:** all cryptography uses the standard Java Cryptography Architecture (no exotic libraries); Spring Boot and React are mature and well documented.
- **Economic:** entirely open-source stack; runs on a student laptop with zero licence cost.
- **Operational:** users only need a browser; verifiers do not even need an account.

4 System Design

4.1 Architecture

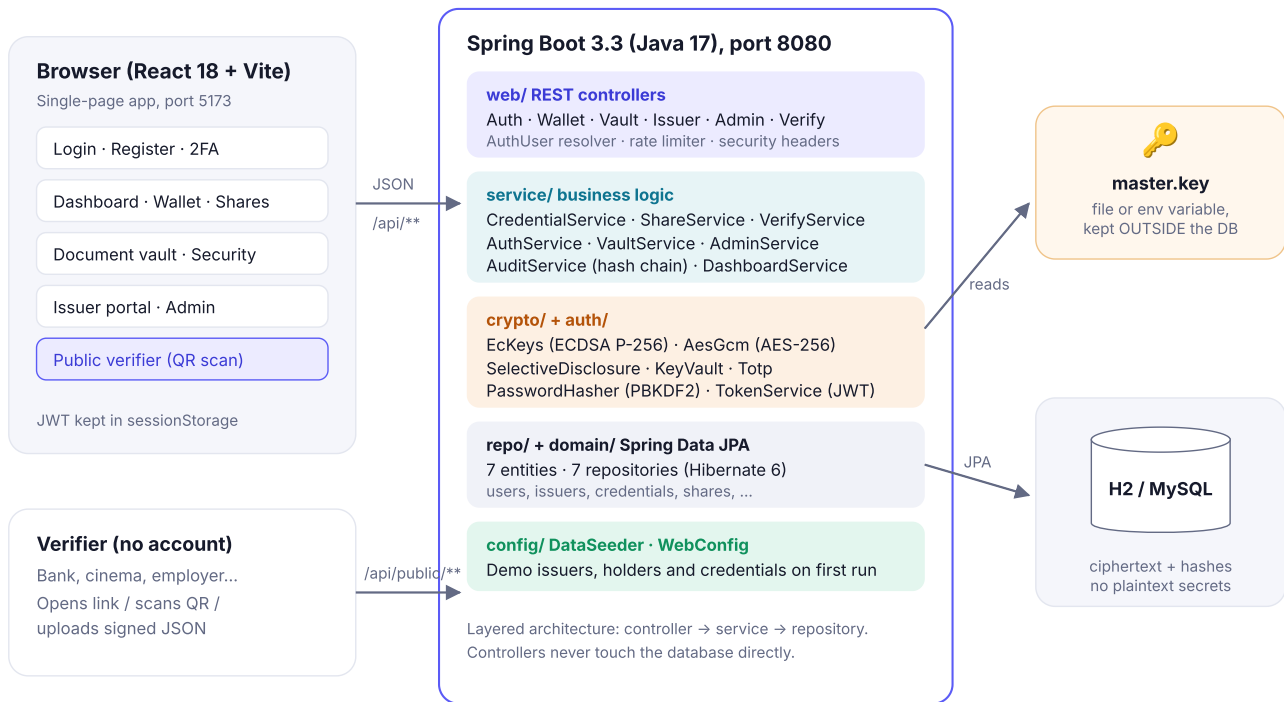


Figure 4.1 — Three-tier architecture of IdentityWallet

The system follows a **three-tier, layered architecture**:

- **Presentation tier:** a React 18 single-page application (Vite dev server on port 5173). It talks to the backend only through JSON REST APIs under `/api`, and keeps the login token in `sessionStorage` so it disappears when the tab closes.
- **Application tier:** a Spring Boot 3.3 application (port 8080) organised in packages — `web` (controllers), `service` (business logic), `crypto` and `auth` (security primitives), `repo` and `domain` (persistence), `config` (setup).
- **Data tier:** H2 or MySQL accessed through Spring Data JPA / Hibernate. Sensitive columns contain only ciphertext or hashes. The **master key** lives outside the database (file or environment variable).

4.2 Modules

Module	Responsibility	Main classes
Authentication	Register, login, 2FA, logout, JWT sessions, password change	<code>AuthService</code> , <code>TokenService</code> , <code>PasswordHasher</code> , <code>Totp</code> , <code>RateLimiter</code>
Credential management	Issue, list, view, revoke, remove credentials	<code>CredentialService</code> , <code>CredentialTemplates</code> , <code>SelectiveDisclosure</code>

Module	Responsibility	Main classes
Sharing & consent	Build signed presentations, links, QR, view limits, access logs	ShareService , AccessLog
Verification	Run the five verification checks for links and uploaded files	VerifyService , VerifyController
Document vault	Encrypted documents with file-type checking and integrity hash	VaultService , AesGcm , KeyVault
Administration	Trust registry, user management, statistics	AdminService
Audit	Hash-chained event log and chain verification	AuditService
Dashboard	Summary, expiring items, security score	DashboardService

4.3 Credential lifecycle

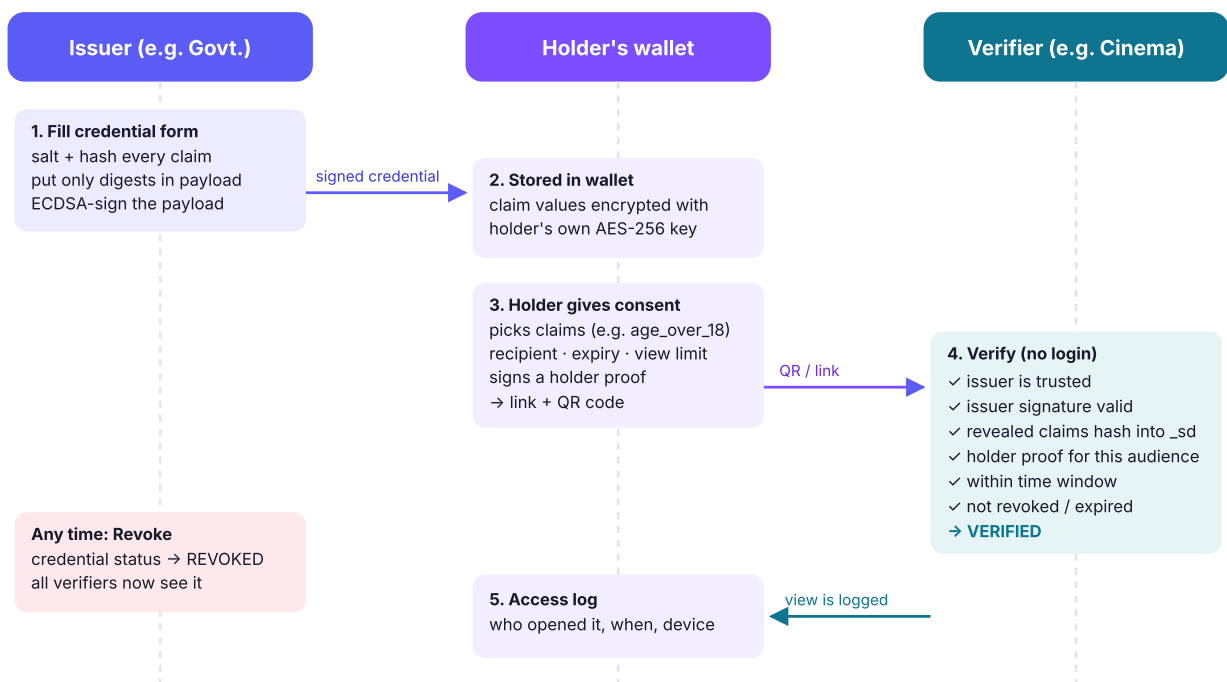
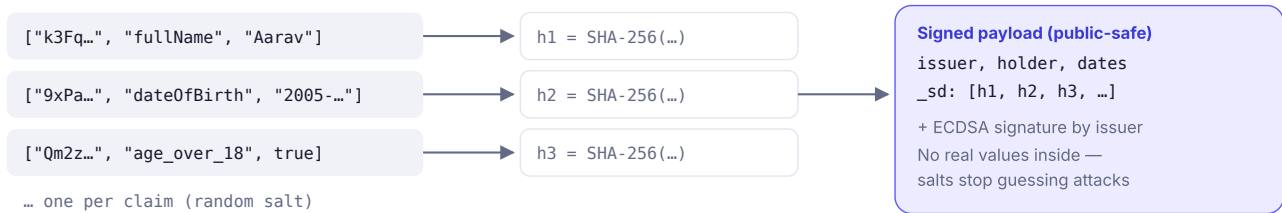


Figure 4.2 — Issue → hold → share → verify → log → revoke

4.4 Selective disclosure design

A. Issuance — each claim becomes a salted disclosure; only its hash is signed



B. Presentation — holder reveals only what is needed

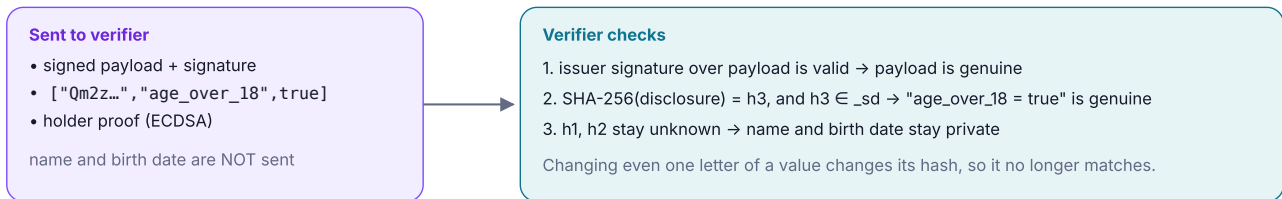


Figure 4.3 — Only salted hashes are signed; the holder reveals a subset

For every claim the issuer creates a **disclosure** — the JSON array `[salt, name, value]`, Base64url-encoded — and computes its **digest** `Base64url(SHA-256(disclosure))`. The signed credential payload contains only the sorted list of digests (`_sd`). To share, the holder sends the payload, the issuer's signature and only the chosen disclosures. The verifier hashes each received disclosure and checks that the digest appears in the signed list.

Because each disclosure has a 128-bit random salt, a verifier cannot guess hidden values by hashing every possible date of birth. Because the payload is signed, the holder cannot invent new claims or change values.

4.5 Key management (envelope encryption)

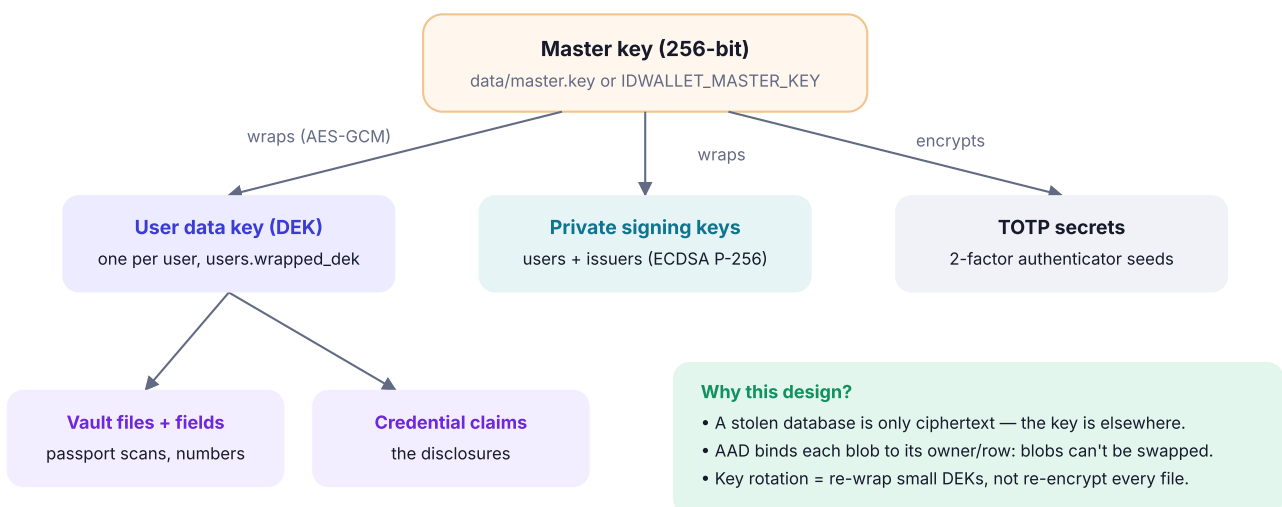


Figure 4.4 — Envelope encryption: one master key wraps per-user keys

4.6 Database design

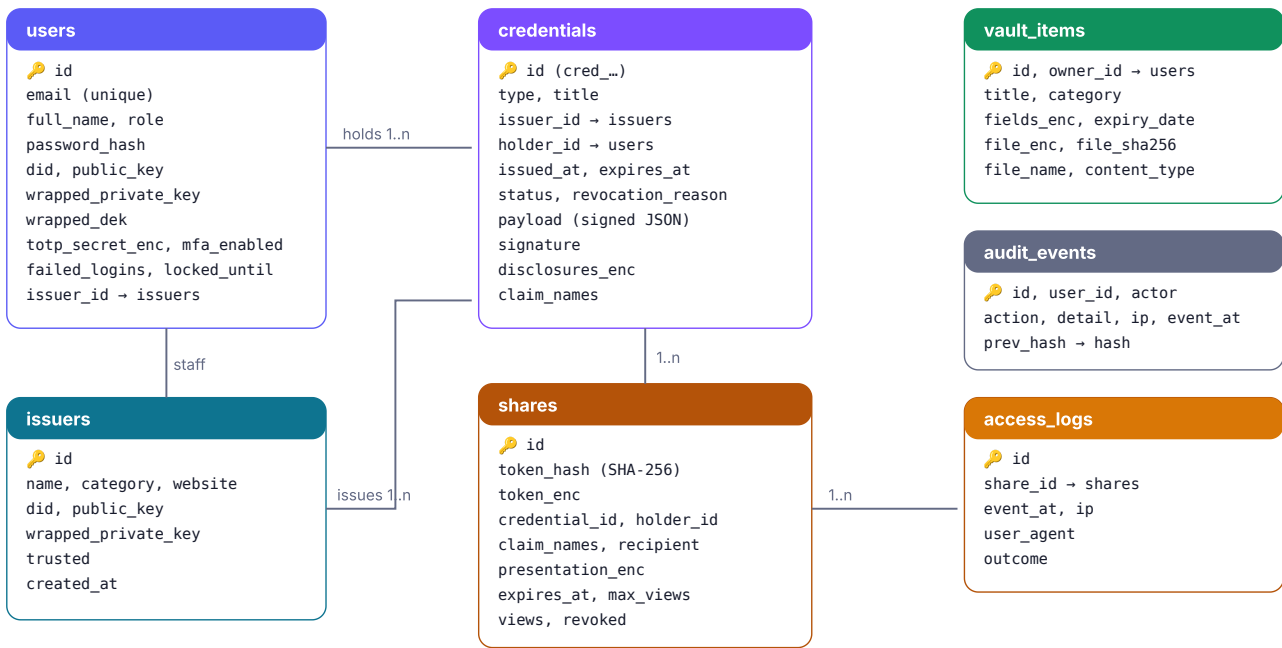


Figure 4.5 — Entity-relationship diagram

Table	Purpose	Sensitive data protection
users	Accounts for holders, issuer staff and admins	PBKDF2 password hash; private key, data key and TOTP secret encrypted
issuers	Trust registry of organisations	Private signing key encrypted with the master key
credentials	Signed credentials held in wallets	Payload holds only digests; claim values encrypted with holder's key
shares	Consent-based shares	Only SHA-256 of link token stored; presentation encrypted with a token-derived key
access_logs	Every opening of a share link	IP and device for the holder's transparency
vault_items	Personal documents	Fields and file encrypted (AES-256-GCM) + SHA-256 integrity hash
audit_events	Security event log	Each row stores previous row's hash (tamper-evident chain)

4.7 REST API design

Method	Endpoint	Access
POST	/api/auth/register , /api/auth/login , /api/auth/mfa	Public (rate-limited)
GET	/api/auth/me , /api/auth/activity	Signed-in user

Method	Endpoint	Access
POST	<code>/api/auth/mfa/setup</code> , <code>/mfa/confirm</code> , <code>/mfa/disable</code> , <code>/password</code>	Signed-in user
GET	<code>/api/wallet/dashboard</code> , <code>/credentials</code> , <code>/credentials/{id}</code> , <code>/shares</code>	Holder
POST	<code>/api/wallet/shares</code> , <code>/api/wallet/shares/{id}/revoke</code>	Holder
GET/POST/DELETE	<code>/api/vault</code> , <code>/api/vault/{id}</code> , <code>/api/vault/{id}/file</code>	Holder
GET/POST	<code>/api/issuer/templates</code> , <code>/credentials</code> , <code>/credentials/{id}/revoke</code>	Issuer
GET/POST	<code>/api/admin/stats</code> , <code>/issuers</code> , <code>/users</code> , <code>/audit</code> , <code>/audit/verify</code>	Admin
GET/POST	<code>/api/public/share/{token}</code> , <code>/api/public/verify</code> , <code>/api/public/issuers</code>	Anyone (rate-limited)

5 Implementation

5.1 Technology stack

Layer	Technology	Why it was chosen
Backend language	Java 17	Strong typing, mature built-in cryptography (JCA)
Framework	Spring Boot 3.3 (Web, Data JPA, Validation)	Rapid REST development, dependency injection, transactions
ORM / DB	Hibernate 6, H2 (dev) / MySQL 8 (prod)	Zero-setup development, production-ready deployment
Frontend	React 18, React Router 6, Vite 5	Component-based, fast, interactive UI
UI libraries	lucide-react (icons), qrcode.react (QR), jsQR (camera scanning)	Lightweight and well maintained
Testing	JUnit 5, Spring MockMvc	Unit + full HTTP flow testing

5.2 Project structure

```
identity-wallet/
├── backend/ (Spring Boot)
│   ├── src/main/java/com/cecil/idwallet/
│   │   ├── crypto/  AESGcm, EcKeys, KeyVault, SelectiveDisclosure, Totp, CryptoUtil
│   │   ├── auth/    PasswordHasher, TokenService, AuthUser, RateLimiter
│   │   ├── domain/  7 JPA entities
│   │   ├── repo/    7 Spring Data repositories
│   │   ├── service/ Auth, Credential, Share, Verify, Vault, Admin, Dashboard, Audit
│   │   ├── web/     REST controllers + global error handler
│   │   └── config/  WebConfig, DataSeeder
└── frontend/ (React)
    ├── src/pages/  Login, Dashboard, Wallet, CredentialPage, Shares,
                    Vault, Security, IssuerPortal, Admin, Verify
```

5.3 Issuing a signed credential

When an issuer submits a form, `CredentialService.issue()` validates the claims, adds derived age claims, creates one salted disclosure per claim, signs the payload and encrypts the disclosures with the holder's key:

```

List<Disclosure> disclosures = claims.entrySet().stream()
    .map(e -> SelectiveDisclosure.create(e.getKey(), e.getValue()))
    .toList();
List<String> digests = disclosures.stream().map(Disclosure::digest).sorted().toList();

payload.put("issuer", Map.of("did", issuer.did, "name", issuer.name));
payload.put("holder", holder.did);
payload.put("_sd_alg", "sha-256");
payload.put("_sd", digests); // only hashes are signed
c.payload = json.writeValueAsString(payload);
c.disclosuresEnc = AesGcm.encryptString(userService.dek(holder),
    json.writeValueAsString(encodedDisclosures), "cred:" + c.id);
c.signature = EcKeys.sign(issuerPrivateKey, c.payload); // ECDSA P-256

```

Derived claims: if a date of birth is present, the issuer also signs `age_over_18` and `age_over_21` booleans computed at issuance, so the holder can later prove age without revealing the date.

5.4 Creating a share (holder consent)

`ShareService.create()` selects only the requested disclosures and signs a **holder proof** that binds them to the recipient and time window, preventing replay to a different verifier:

```

String sig = EcKeys.sign(holderPrivateKey,
    proofMessage(c.payload, chosenDisclosures, recipient, nonce, createdAt, expiresAt));

s.tokenHash = CryptoUtil.sha256Hex(token); // only the hash is stored
s.presentationEnc = AesGcm.encryptString(
    sha256("idwallet-share-key|" + token), presentationJson, "share");

```

The random 192-bit link token never touches the database: only its SHA-256 hash is stored, and the presentation is encrypted with a key derived from the token itself. A database leak therefore reveals neither shared data nor working links.

5.5 Verification

`VerifyService.verifyPresentation()` performs five independent checks and reports each one with a reason:

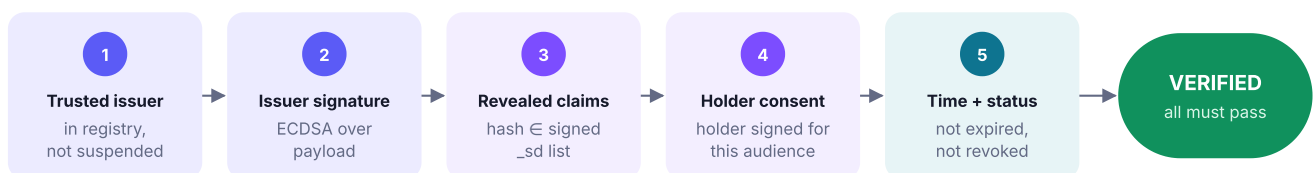


Figure 5.1 — The verification pipeline

```
boolean sigOk = EcKeys.verify(issuerPublicKey, payload, issuerSignature);
for (JsonNode n : vp.path("disclosures")) {
    Disclosure d = SelectiveDisclosure.decode(n.asText());
    allMatch &= signedDigests.contains(d.digest()); // value not altered
}
boolean proofOk = EcKeys.verify(holderPublicKey, proofMessage(...), proofSignature);
```

5.6 Encryption and key management

- `AesGcm` implements AES-256-GCM with a fresh 12-byte IV per encryption and a 128-bit authentication tag; **AAD** (additional authenticated data) such as `"vault-file:42"` binds each ciphertext to its record.
- `KeyVault` loads the 256-bit master key from `IDWALLET_MASTER_KEY` or `data/master.key` and wraps/unwraps user data keys, private keys and TOTP secrets.
- Vault uploads are type-checked by **magic bytes** (`%PDF-`, PNG, JPEG, WEBP signatures), not by the file name, and verified with SHA-256 on every download.

5.7 Authentication and session security

- **Passwords:** PBKDF2-HMAC-SHA256, 16-byte random salt, **210,000 iterations**, constant-time comparison; policy of 8+ characters with letter, digit and symbol.
- **Sessions:** HS256-signed JWT with expiry and a `kind` claim, so a 5-minute MFA ticket can never be used as a full session.
- **Two-factor:** RFC 6238 TOTP (30-second codes) compatible with Google/Microsoft Authenticator; secrets encrypted at rest.
- **Brute force:** 5 wrong passwords lock the account for 5 minutes; per-IP rate limits on login, MFA, sign-up and verification; identical errors and a dummy hash for unknown emails prevent account enumeration.
- **Authorisation:** an `AuthUser` argument resolver requires a valid token; `require(Role...)` checks roles; ownership checks return 404 for other people's records (IDOR protection).

5.8 Tamper-evident audit log

Every event stores `hash = SHA-256(prevHash | user | actor | action | detail | ip | time)`. Editing or deleting any row breaks every later hash, which `AuditService.verifyChain()` detects and the admin panel displays. A lock is held until the event's own transaction commits, so two concurrent events can never fork the chain.

5.9 Frontend highlights

- Role-based navigation (holder, issuer, admin) and route guards.
- Credential cards styled per type; masked values with a *Reveal* toggle.
- Share dialog with presets (*Age check only*, *Name + age*, *Everything*), live count of hidden details, expiry and view-limit selectors.
- QR generation (`qrcode.react`) and camera-based QR scanning (`jsQR`) on the verifier page.

- Security score with an explainable checklist; responsive layout with automatic dark mode.

6 Security Analysis

6.1 Threat model (STRIDE)

Threat	Example attack	Mitigation in IdentityWallet
Spoofing	Attacker logs in as a user	PBKDF2 hashing, 2FA (TOTP), logout, rate limiting
Spoofing	Fake issuer creates credentials	Trust registry; verifier only accepts registered, non-suspended issuer keys
Tampering	Holder edits "age_over_18" to true	Digest no longer matches the signed <code>_sd</code> list → "Revealed details" fails
Tampering	Attacker edits the credential payload	ECDSA signature check fails
Tampering	Admin/DBA edits audit rows	Hash chain breaks; detected by chain verification
Repudiation	User denies sharing data	Holder proof signed with the holder's private key + access log + audit log
Information disclosure	Database is stolen	Only ciphertext and hashes; master key stored separately
Information disclosure	Verifier guesses hidden birth date	128-bit random salt per disclosure makes guessing infeasible
Information disclosure	User opens another user's credential by ID	Ownership check returns 404 (IDOR protection)
Denial of service	Password or verification flooding	Per-IP sliding-window rate limits, upload size limit (5 MB)
Elevation of privilege	Holder calls issuer/admin API	Role checks on every protected endpoint (403)
Elevation of privilege	MFA ticket used as session	Token <code>kind</code> claim checked; tickets expire in 5 minutes

6.2 OWASP Top 10 mapping

OWASP 2021	Handling
A01 Broken Access Control	Token required, role checks, ownership checks, 404 for foreign IDs
A02 Cryptographic Failures	AES-256-GCM, ECDSA P-256, PBKDF2 210k, SHA-256, secure random, no custom algorithms
A03 Injection	JPA parameterised queries only; claim names validated by regex
A04 Insecure Design	Privacy by design: data minimisation, consent, expiry, view limits

OWASP 2021	Handling
A05 Security Misconfiguration	Security headers (<code>nosniff</code> , <code>X-Frame-Options</code> , <code>no-referrer</code> , <code>no-store</code>), no stack traces to clients
A07 Identification & Authentication Failures	2FA, logout, rate limits, password policy, generic error messages
A08 Software & Data Integrity Failures	Signed credentials, hash-chained audit, SHA-256 integrity on files
A09 Logging & Monitoring Failures	Audit log of logins, failures, shares, views, revocations, admin actions

7 Testing

7.1 Test strategy

Testing was done at two levels: **unit tests** for the cryptographic building blocks (`CryptoTest`) and **integration tests** that run the whole Spring Boot application with an in-memory database and call the real REST APIs through MockMvc (`WalletFlowTest`). Manual UI testing covered every page on desktop and mobile.

7.2 Automated test cases

#	Test	Input / action	Expected result	Result
1	TOTP RFC 6238 vector	Secret "12345678901234567890", T = 59 s	Code 287082	Pass
2	AES-GCM round trip & tamper	Encrypt, decrypt; then wrong AAD and flipped byte	Original text; then exception	Pass
3	ECDSA sign / verify	Sign "hello"; verify "hello"	Valid; then invalid	Pass
4	PBKDF2 hashing	Hash same password twice; check wrong password	Different hashes; wrong rejected	Pass
5	Disclosure digest	Create, encode, decode a disclosure	Same digest; new salt gives new digest	Pass
6	Full flow	Issue → share 2 claims → verify	Valid; 2 shown, 5 hidden, ID number not leaked	Pass
6a	Forged claim	Replace name disclosure with "Someone Else"	Verification fails	Pass
6b	View limit	Open a 2-view link three times	Third attempt refused	Pass
6c	Revocation	Revoke, then verify the saved presentation	Fails "Credential status"	Pass
6d	Access control	Holder calls issuer API; user reads another's credential; no token	403, 404, 401	Pass
6e	Audit chain	Verify chain after the flow	Chain intact	Pass
7	Vault	Upload PDF, download; upload HTML renamed .pdf	Same bytes; fake file rejected (400)	Pass
8	MFA and logout	Enable 2FA; login; wrong code; right code; 5 wrong passwords	Ticket only; 401; session; account locked (423)	Pass

7.3 Test execution result

```
[INFO] Running com.cecil.idwallet.CryptoTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.cecil.idwallet.WalletFlowTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

7.4 Manual test summary

Scenario	Observed
Login with each demo role	Correct dashboard per role
Share "Age check only" with City Cinema	Verifier sees only "Age over 18: Yes"; 7 details hidden; all checks green
Open link beyond view limit / after "Stop sharing"	Refused with a clear reason; attempt appears in access log
Enable 2FA with Google Authenticator	Sign-in requires the 6-digit code
Issuer suspends / restores	All credentials of that issuer fail / pass verification
Mobile width (390 px)	Layout adapts; sidebar becomes a menu

8 Results and Screenshots

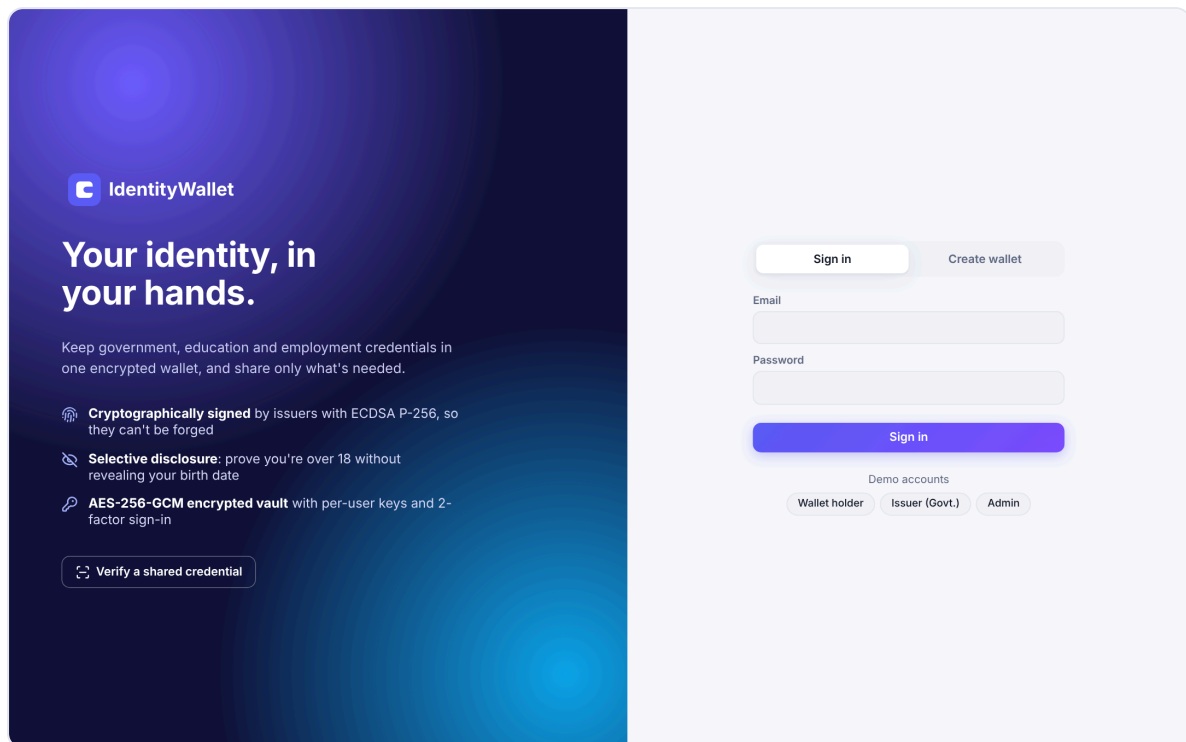


Figure 8.1 — Sign-in page with demo accounts

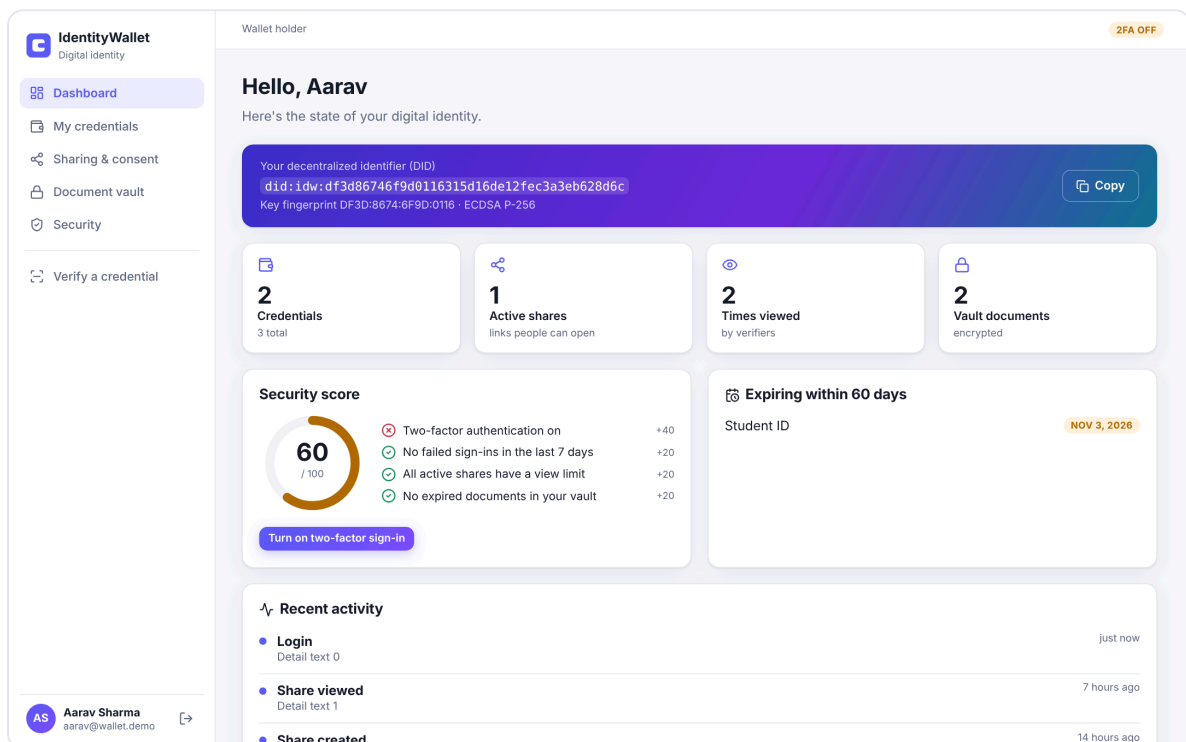


Figure 8.2 — Holder dashboard: DID, statistics, security score, activity

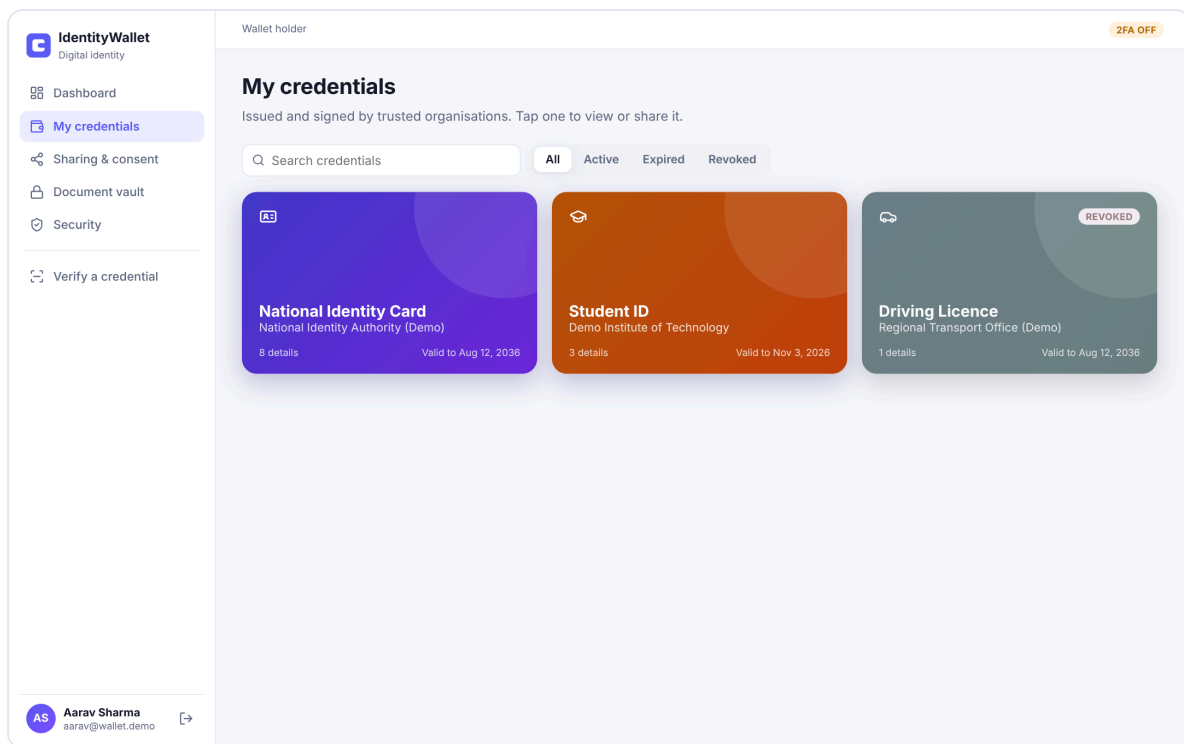


Figure 8.3 — Wallet with active and revoked credentials

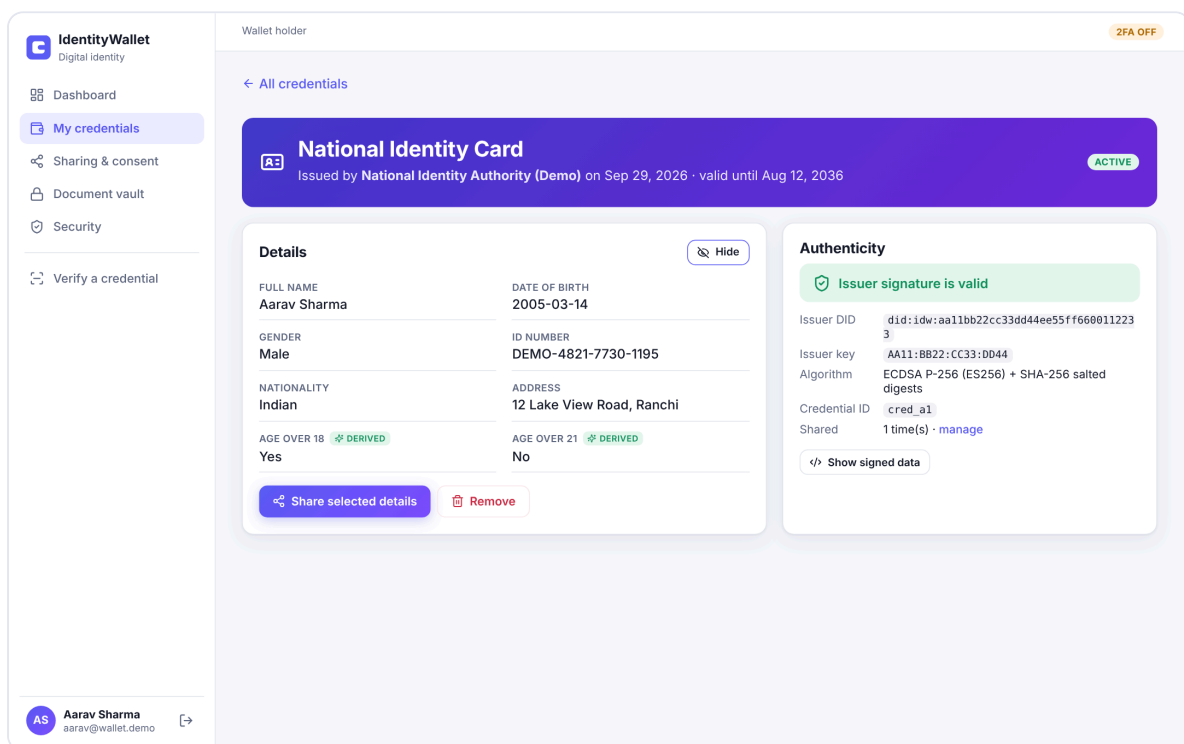


Figure 8.4 — Credential details and authenticity panel

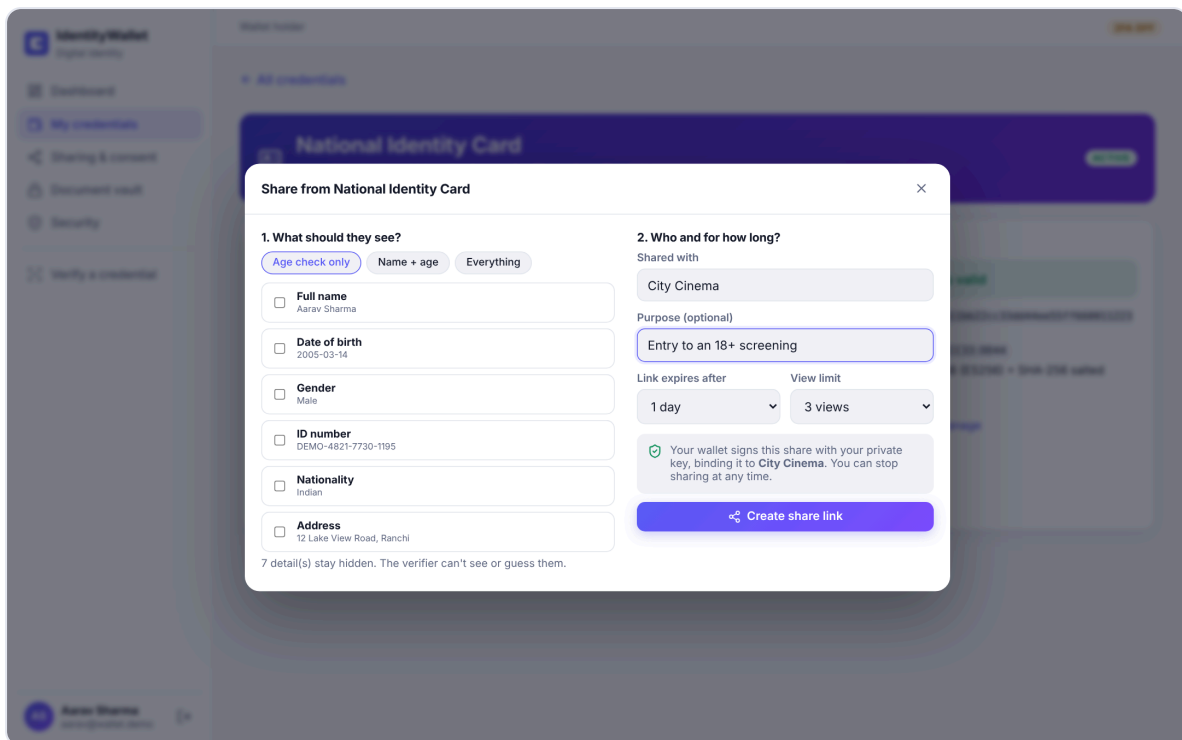


Figure 8.5 — Consent-based share dialog (selective disclosure)

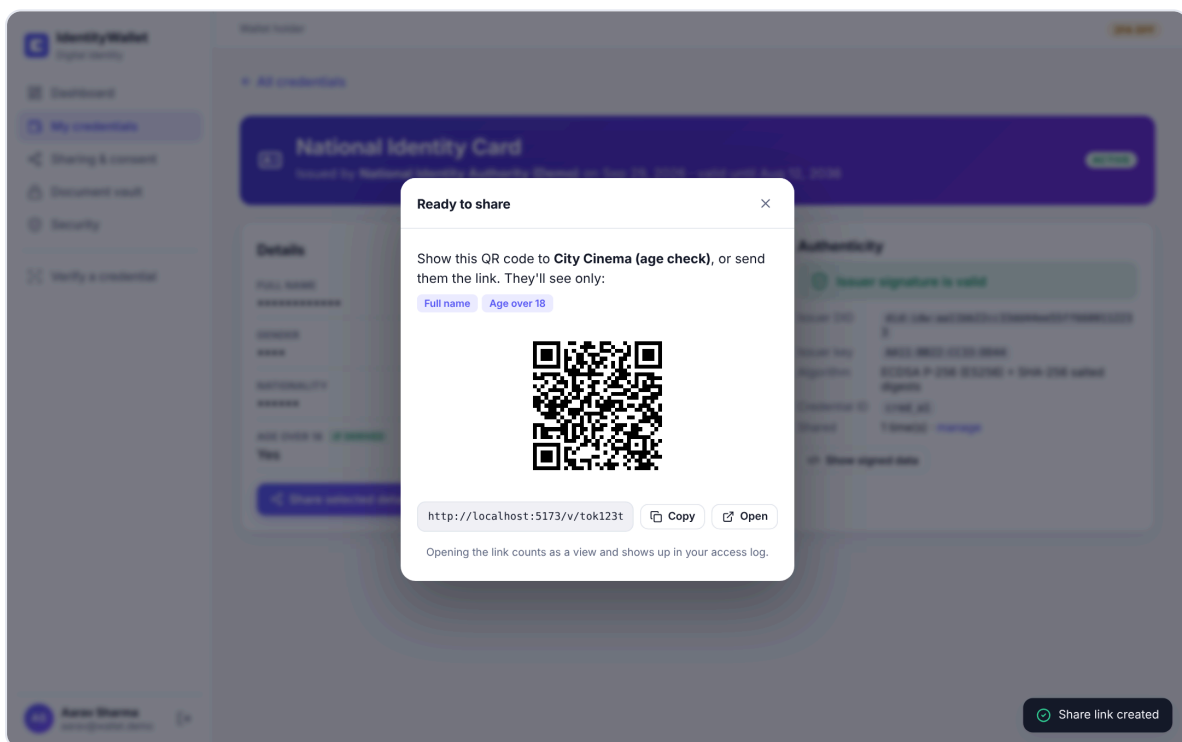


Figure 8.6 — Share link and QR code

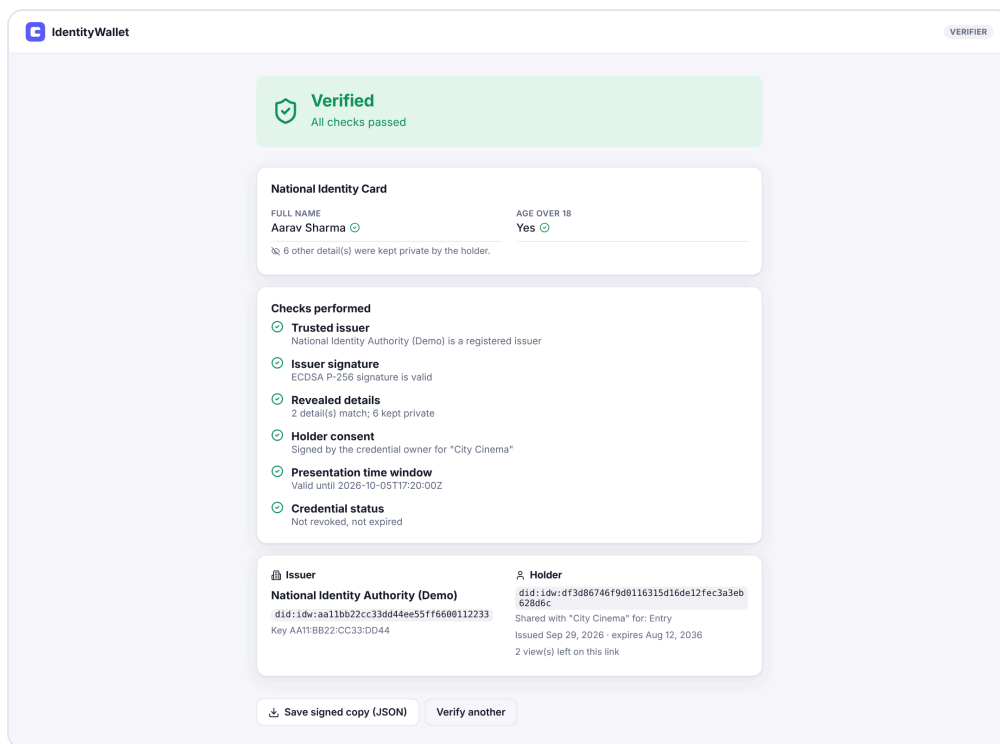


Figure 8.7 — Verifier result: only "Age over 18" revealed, all checks passed

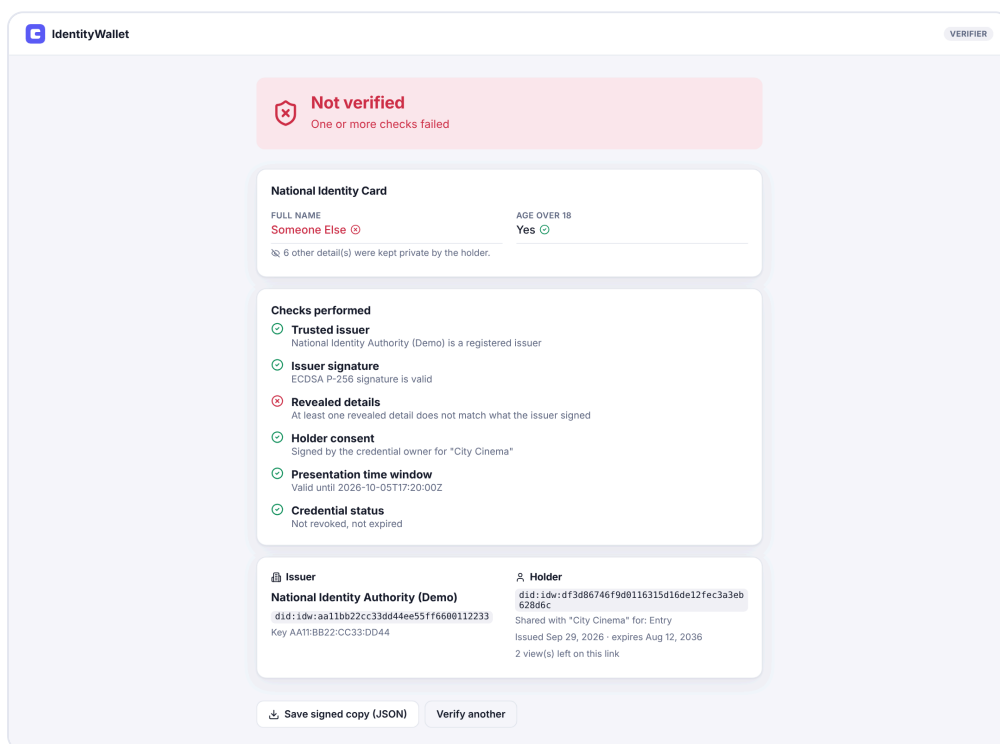


Figure 8.8 — A tampered presentation is rejected

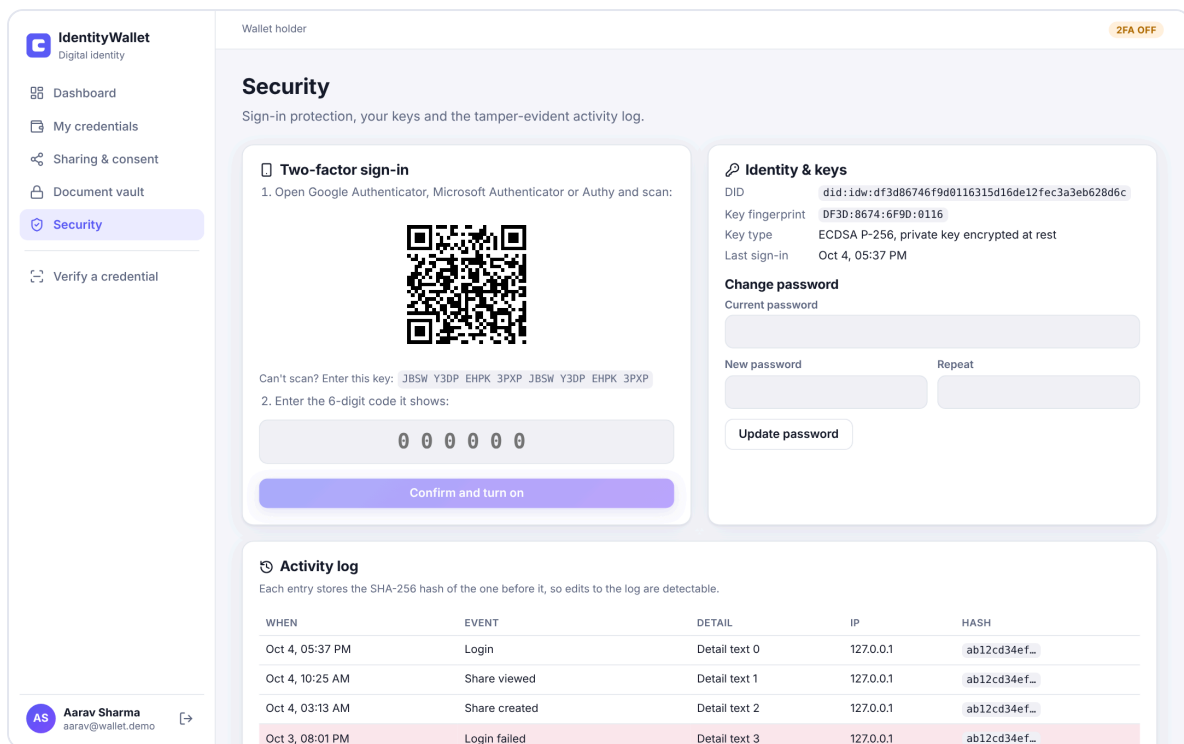


Figure 8.11 — Two-factor authentication setup

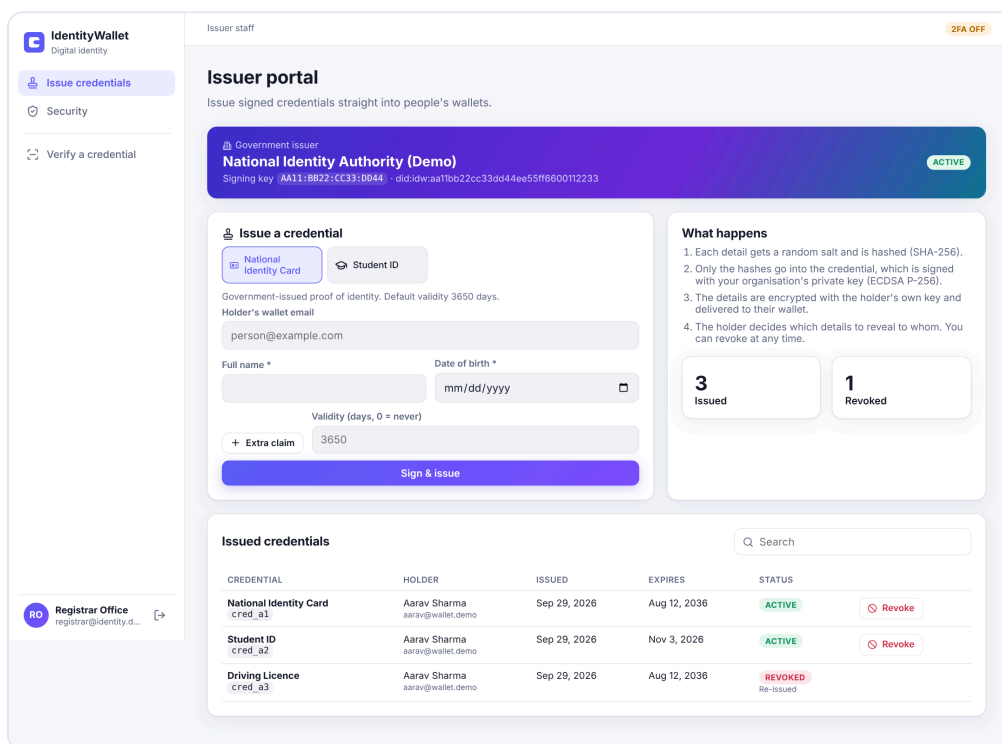


Figure 8.12 — Issuer portal

IdentityWallet

Digital identity

Administration

Security

Verify a credential

PA

Platform Admin

admin@idwallet.local

Administrator

2FA OFF

Administration

Trust registry, accounts and the platform audit trail.

2

Wallet holders

3

Issuers

4

Credentials

1

Revoked

1

Shares

Audit chain intact

23 events

Trust registry

Users

Audit log

Platform audit log (latest 200)

Verify hash chain

#	WHEN	ACTOR	EVENT	DETAIL	IP	HASH
10	Oct 4, 05:37 PM	aarav@wallet.demo	Login	Detail text 0	127.0.0.1	ab12cd34ef...
9	Oct 4, 10:25 AM	aarav@wallet.demo	Share viewed	Detail text 1	127.0.0.1	ab12cd34ef...
8	Oct 4, 03:13 AM	aarav@wallet.demo	Share created	Detail text 2	127.0.0.1	ab12cd34ef...
7	Oct 3, 08:01 PM	aarav@wallet.demo	Login failed	Detail text 3	127.0.0.1	ab12cd34ef...
6	Oct 3, 12:49 PM	aarav@wallet.demo	Credential received	Detail text 4	127.0.0.1	ab12cd34ef...
5	Oct 3, 05:37 AM	aarav@wallet.demo	Registered	Detail text 5	127.0.0.1	ab12cd34ef...

Figure 8.13 — Admin panel: hash-chained audit log

IdentityWallet — Project Report · Cecil Baraik

30 / 32

9 Conclusion and Future Scope

9.1 Conclusion

IdentityWallet demonstrates that digital identity can be **both verifiable and private**. Issuers sign credentials that cannot be forged; holders keep them encrypted and decide exactly which facts to share, with whom, for how long and how many times; verifiers get a clear, automatic answer in under a second without needing an account. The project applies real-world cryptography (ECDSA, AES-GCM, SHA-256, PBKDF2, TOTP) and secure design principles (least privilege, data minimisation, defence in depth, tamper evidence) in a complete, tested full-stack application. All 8 automated tests — including forgery, tampering, replay and brute-force scenarios — pass.

9.2 Limitations

- Private keys are custodial (held encrypted on the server) rather than on the user's device.
- The credential format is SD-JWT *style* JSON, not the exact standard wire format, so it does not interoperate with other wallets yet.
- Revocation is checked live against the database; verifiers need connectivity.
- Rate limits are in memory, suitable for a single server.

9.3 Future scope

1. **Mobile wallet app** with keys in the phone's secure enclave and biometric unlock; **passkeys (WebAuthn)** for login.
2. Adopt the **IETF SD-JWT VC** and **W3C VC 2.0** formats and OpenID for Verifiable Presentations for interoperability.
3. **Bitstring status lists** for privacy-preserving, offline-friendly revocation checks.
4. **Zero-knowledge proofs** (e.g. BBS+ signatures) so even repeated presentations cannot be linked.
5. **Recovery codes**, TOTP replay protection and email alerts for new sign-ins.
6. **Redis-backed rate limiting**, Docker deployment and HTTPS with a reverse proxy.
7. Integration with national digital-identity ecosystems (e.g. DigiLocker-style issuers) using official APIs.

References

1. W3C, *Verifiable Credentials Data Model v2.0*, World Wide Web Consortium.
2. IETF, *Selective Disclosure for JWTs (SD-JWT)*, Internet-Draft, OAuth Working Group.
3. W3C, *Decentralized Identifiers (DIDs) v1.0*.
4. NIST, *FIPS 186-5: Digital Signature Standard (DSS)*.
5. NIST, *SP 800-38D: Recommendation for Block Cipher Modes of Operation — GCM and GMAC*.
6. IETF, *RFC 6238: TOTP — Time-Based One-Time Password Algorithm*.
7. IETF, *RFC 8018: PKCS #5 — Password-Based Cryptography Specification (PBKDF2)*.
8. IETF, *RFC 7519: JSON Web Token (JWT)*.
9. OWASP Foundation, *OWASP Top 10 (2021)* and *Password Storage Cheat Sheet*.
10. Government of India, *Digital Personal Data Protection Act, 2023*.
11. Spring, *Spring Boot 3.3 Reference Documentation*.
12. Meta Open Source, *React 18 Documentation*.